Usage notes

Regenerating the artifacts I

From the monorepo root, regenerate the figures and the release-review artifacts:

```
uv run python projects/templates/template_data_descriptor/s
uv run python projects/templates/template_data_descriptor/s
```

The first writes the five figures under `output/figures/`; the second writes the descriptor readiness report, field-constraint summary, and metadata-only release manifest under `output/reports/`. Both are thin orchestrators: all computation lives in the tested `src/data_descriptor/` package.

Forking in a real dataset I

To adapt this template to a real dataset:

Forking in a real dataset I

1. Replace the fixture files under `data/fixtures/` with your data files (or, for restricted data, leave them out and publish descriptor + checksums only).
2. Edit `data/example_descriptor.json`: update `name`, `license`, the `files` inventory (path, media type, sha256 checksum, row count), the `fields` data dictionary, `primary_key`, and the `provenance` chain.
3. Recompute each file's checksum and row count and record them in the descriptor. For present files, `verify_descriptor_files()` will confirm the descriptor matches the bytes; a mismatch fails the test suite.
4. Extend field constraints (patterns, enumerations, numeric bounds, units) before publishing real data, and keep the provenance chain honest — cite real tools, versions, and operators.

Forking in a real dataset I

Use `scripts/audit/copy_exemplar.py` from the monorepo to fork the template cleanly, and keep `domain_profile.yaml` and `experiment_plan.yaml` aligned with your dataset.

Boundaries and claims I

This is a template with a synthetic demonstration dataset. Its claims are limited to dataset structure, schema completeness, provenance depth, license state, byte-level integrity, and release readiness — never to any empirical effect in the data. When you fork it, keep that discipline: a data descriptor earns trust by describing a dataset precisely and verifiably, not by over-claiming what the dataset shows.